High-level idea

The code finds a path from `startNode` to `endNode` that minimizes the *Blackie-length* — a path cost where the cost of each edge depends on whether its position in the path is a prime index.

It does this in **three phases**:

1. Precompute primes and a **position-independent heuristic** using Dijkstra on a "static" edge cost `min(w1, 3*w2)`.
2. Run several **cheap approximate Dijkstras** with different weight combinations to get a good initial solution and upper bound.
3. Use a **time-bounded, heuristic-guided A*** (with pruning and cycle control) that accounts for the **true position-dependent cost**, improving the initial path when possible.

Global variables

- `int N, M, startNode, endNode;`
  - `N` : number of nodes.
  - `M` : number of edges.
  - `startNode` : source vertex id.
  - `endNode` : target vertex id.
- `vector<vector<Edge>> graph;` Adjacency list representation of the undirected graph. For each node, stores a list of outgoing edges (`Edge` structs). Needed by all search algorithms.
- `vector<bool> is_prime;` Precomputed boolean array where `is_prime[i]` tells whether `i` is prime. Used to decide whether the i-th edge in a path uses `w1` or `3*w2` cost.
- `vector<long long> h_cost_to_target;` Heuristic: for each node `u`, minimal possible "static cost" from `u` to `endNode` using `min(w1, 3*w2)` per edge. Computed by reverse Dijkstra, used in A* as an admissible heuristic for pruning.
- `vector<int> global_best_path;` The best path found so far (sequence of node IDs). Shared between seeding and A* phase.
- `long long global_best_cost = numeric_limits<long long>::max();` The Blackie-length of `global_best_path`. Used as an upper bound to prune worse states in A*.
- `vector<tuple<int, int>> path_storage;` Stores compressed path information for A*: each entry is `(node, parent_idx)`.
  - `node` : the vertex at this step.
  - `parent_idx` : index of the previous step in `path_storage` (or −1 for root). Enables cheap path reconstruction without storing full paths in the priority queue.
- `unordered_map<long long, long long> min_cost_for_state;` Dominance/pruning table. Key encodes `(node, steps)`; value is the minimum `g_cost` seen for that state so far. Used to prune A* states that are strictly worse than an already explored state with same `(node, steps)`.
- `const long long INF = numeric_limits<long long>::max() / 3;` Large number treated as "infinite" distance; using `/3` to avoid overflow when summing.
- `auto t_start = chrono::steady_clock::now();` Start timestamp of the program, used for runtime checks.
- `const double TIME_LIMIT_TOTAL = 19.0 [174.0 for long];` Time budget (seconds) for the entire computation, below the problem's 20s [180s for long] limit for safety.
- `const double TIME_LIMIT_SEEDING = 5.0 [140.0 for long];` Time budget (seconds) reserved for the seeding phase (the approximate Dijkstras).
- `long long loop_counter = 0;` Counts iterations in the A* search loop; used to periodically check the time.
- `bool time_limit_exceeded = false;` Flag set when the total time budget is exceeded, so that further heavy work is skipped.

Structs

`struct Edge`

- **Purpose**: Represents an edge in the graph.
- **Fields**:
  - `to` : destination node.
  - `w1` : first weight (used when edge index is not prime).
  - `w2` : second weight (used with factor 3 when edge index is prime).

`struct StateAlpha`

- **Purpose**: Node state for the **alpha-weighted Dijkstra** (`dijkstra_alpha`).
- **Fields**:

- `node` : current node.
- `dist` : current accumulated approximate distance using `w1 + alpha * w2` .

`struct StateStatic`

- **Purpose**: State for Dijkstra on **static costs** (min of `w1` and `3*w2`) and for the heuristic precomputation.
- **Fields**:
  - `node` : current node.
  - `dist` : accumulated static distance from source (or to target in reverse Dijkstra).

`struct StateAStar`

- **Purpose**: State in the **main A* search** (`find_path_iterative`), which respects position-dependent costs.
- **Fields**:
  - `f_cost` : estimated total cost = `g_cost + w * heuristic` , used as priority.
  - `g_cost` : exact cost so far (true Blackie-length accumulated).
  - `node` : current node.
  - `steps` : number of edges taken so far; also the index for the next edge.
  - `parent_idx` : index into `path_storage` for path reconstruction.
- **Comparison operator**:
  - First compares `f_cost` (lower is better).
  - Breaks ties by preferring **more steps** (deeper states), pushing towards deeper/longer paths when f-costs are equal.

Functions

`void sieve_primes(int n)`

- **What it does**: Computes a prime sieve up to `n` using the classic O(n log log n) approach, setting `is_prime[i]` to true iff `i` is prime.

`inline long long static_cost(const Edge &e)`

- **What it does**: Returns `min(w1, 3 * w2)` for a given edge.

`inline long long step_cost(const Edge &e, int step_index)`

- **What it does**: Returns the **true** cost of taking edge `e` at position `step_index` :
  - If `step_index` is prime → `3 * w2` .
  - Else → `w1` .

`long long calculate_blackie_length(const vector<int> &path)`

- **What it does**:
  - Given a node sequence `path` , iterates over consecutive pairs `(u, v)` .
  - For each pair, finds the corresponding edge in `graph[u]` .
  - Adds `step_cost(edge, i+1)` for edge index `i+1` .
  - Returns INF if any edge `(u,v)` is missing.

`vector<int> reconstruct_path(int final_parent_idx)`

- **What it does**:
  - Rebuilds a path of nodes by walking backwards through `path_storage` using `parent_idx` until `-1` is reached.
  - Reverses the collected sequence to get start → end order.

`inline long long encode_state(int node, int steps)`

- **What it does**:
  - Encodes `(node, steps)` into a 64-bit integer:

    ```
    return ((long long)node << 32) | (steps);
    ```

`bool BoundedCheckCycle(int v, int parent_idx, int max_depth = 20)`

- **What it does**:
  - Walks backwards along the path in `path_storage` following `parent_idx`, up to `max_depth` ancestors.
  - Returns `true` if any of those ancestors' nodes equals `v`, meaning we're about to create a short cycle.

`vector<int> dijkstra_alpha(int src, int dst, double alpha)`

- **What it does**:
  - Runs standard Dijkstra's algorithm from `src` to `dst` on an edge cost:

    ```
    cost = w1 + alpha * w2;
    ```

  - Uses `StateAlpha` with `double dist`.
  - If `dst` is reachable, reconstructs and returns the path; otherwise returns empty vector.

`vector<int> dijkstra_static_path(int src, int dst)`

- **What it does**:
  - Runs Dijkstra's algorithm using `static_cost(e)` on each edge.
  - Returns a shortest path from `src` to `dst` in terms of static cost, or empty if unreachable.

`void calculate_reverse_dijkstra_heuristic(int target)`

- **What it does**:
  - Runs Dijkstra starting from `target` over the graph using `static_cost(e)`.
  - Fills `h_cost_to_target[u]` = minimal static cost from `u` to `target`.

`void find_path_iterative(double heuristic_weight)`

- **What it does** (core A* search):
  1. **Pre-checks**:
     - If `global_best_cost` is INF or `time_limit_exceeded` is true, return immediately.
  2. **Initialization**:
     - Clear `path_storage` and reserve memory.
     - Clear `min_cost_for_state`.
     - Reset `loop_counter`.
     - Push initial state: node = `startNode`, `g_cost = 0`, `steps = 0`, parent index = 0. `f_cost = h_cost_to_target[startNode] * heuristic_weight`.
  3. **Main loop** (while PQ not empty):
     - Pop the state with smallest `f_cost` (and deepest `steps` tie-breaking).
     - Periodically (every 2048 iterations) check elapsed time and set `time_limit_exceeded` if needed.
     - **Prune by bound**: If `f_cost >= global_best_cost`, skip — this path cannot beat the best solution.
     - **Dominance check**: If we have already seen `(node, steps)` with a strictly smaller `g_cost`, skip.
     - **Goal check**:
       - If `node == endNode`:

        - If `g_cost` is better than `global_best_cost`, update `global_best_cost` and reconstruct `global_best_path`.
        - Continue to next state (do not expand neighbors).
    - **Neighbor expansion**:
        - `next_steps = steps + 1`. If `next_steps >= is_prime.size()`, skip (avoid index overflow due to cycles).
        - For each outgoing edge `e` from current node:
            - Let `v = e.to`.
            - If `BoundedCheckCycle(v, parent_idx)` is true, skip to avoid short cycles.
            - Get `h_cost = h_cost_to_target[v]`; if INF, skip (no path to target).
            - Compute `next_edge_cost = step_cost(e, next_steps)` and `ng = g + next_edge_cost`.
            - Compute `nf = ng + heuristic_weight * h_cost`.
            - If `nf >= global_best_cost`, skip (cannot beat current best).
            - Dominance check for `(v, next_steps)` against `min_cost_for_state`; skip if worse or equal.
            - Otherwise:
                - Update `min_cost_for_state` for this state.
                - Append `(v, parent_idx)` to `path_storage`, getting `new_parent_idx`.
                - Push new `StateAStar` with `(nf, ng, v, next_steps, new_parent_idx)` into the PQ.

`main` — Step-by-step description

1. **Fast I/O setup** and store the start time.
2. **Read input**:
    - Number of nodes `N`, edges `M`.
    - `startNode` and `endNode`.
3. **Initialize graph**:
    - `MAX_SIMPLE_PATH_LEN` is used to size the prime sieve (max path length without repeated nodes).
    - Read all edges and fill `graph` as undirected: store both `(u → v)` and `(v → u)`.
4. **Precompute primes** up to `N + 5` to support position-based edge costs.
5. **Compute heuristic (`h_cost_to_target`)**:
    - Reverse Dijkstra from `endNode` using `static_cost(e)`.
6. **Initial seeding (static cost)**:
    - Run `dijkstra_static_path` using `static_cost`.
    - If successful, evaluate this path's true Blackie-length and store it as the initial `global_best_path` and `global_best_cost`.
7. **Aggressive seeding with multiple `alpha` values**:
    - For each `alpha` in the predefined list:
        - Stop if seeding time exceeds `TIME_LIMIT_SEEDING`.
        - Run `dijkstra_alpha` with `cost = w1 + alpha * w2`.
        - If a path is found:
            - Compute its true Blackie-length.
            - If this is better than current `global_best_cost`, update the global best solution.
    - After this step, we hopefully have a **good initial solution**.
8. **No path found in seeding**:
    - If still no path and `startNode == endNode`, answer is trivial: path of length 1 containing only `startNode`.
    - Otherwise, output `0` and an empty line (no path exists), then exit.
    - If a path **was** found, proceed to the next phase.
9. **Iterative A* optimization**:
    - Define a list of heuristic weights from `1.30` (more greedy, less optimal) down to `1.00` (classic A*).
    - For each weight `w`:
        - If time limit has been hit, stop.
        - Call `find_path_iterative(w)` to try to improve the current `global_best_path` using weighted A*.
    - Each run may improve `global_best_cost`, making subsequent runs more aggressive in pruning.

10. **Output the final best path**:
    - Print the number of nodes in `global_best_path`.
    - Print all node IDs in order, space-separated.
    - Exit.

Authors and responsibilities

- **Hubert Plewa**: Responsible for precomputation and utility functions, including the prime sieve (`sieve_primes`), cost calculations (`static_cost`, `step_cost`), path cost evaluation (`calculate_blackie_length`), path reconstruction (`reconstruct_path`) and the reverse Dijkstra heuristic (`calculate_reverse_dijkstra_heuristic`).
- **Łukasz Kowalski**: Responsible for the seeding phase, including implementing the static cost Dijkstra (`dijkstra_static_path`) and the alpha-weighted Dijkstras (`dijkstra_alpha`) with various alpha values to generate initial approximate paths and set the global best solution.
- **Wiktor Smura**: Responsible for implementing state encoding (`encode_state`), cycle detection (`BoundedCheckCycle`), the main A* search implementation (`find_path_iterative`), including state management, pruning, dominance checks, and integrating the iterative weighted A* runs in the main function, as well as overall code structure and time management.